ASSIGNMENT-13.3

Name: M.Rohan

HT. No: 2303A51545

Batch:08

Task 1: Refactoring – Removing Code Duplication

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated

blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Outcome

- A reusable function to calculate area and perimeter
- No duplicated code blocks
- Proper docstrings for all functions

```python
def calculate_rectangle_properties(length, width):
    """
    Calculates the area and perimeter of a rectangle.

    Args:
        length (int or float): The length of the rectangle.
        width (int or float): The width of the rectangle.

    Returns:
        tuple: A tuple containing:
            - area (int or float): The area of the rectangle.
            - perimeter (int or float): The perimeter of the rectangle.
    """
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

# Refactored code using the new function

# Example 1
length1, width1 = 5, 10
area1, perimeter1 = calculate_rectangle_properties(length1, width1)
print(f"Area of Rectangle ({length1}x{width1}):", area1)
print(f"Perimeter of Rectangle ({length1}x{width1}):", perimeter1)

# Example 2
length2, width2 = 7, 12
area2, perimeter2 = calculate_rectangle_properties(length2, width2)
print(f"Area of Rectangle ({length2}x{width2}):", area2)
print(f"Perimeter of Rectangle ({length2}x{width2}):", perimeter2)

# Example 3
length3, width3 = 10, 15
area3, perimeter3 = calculate_rectangle_properties(length3, width3)
print(f"Area of Rectangle ({length3}x{width3}):", area3)
print(f"Perimeter of Rectangle ({length3}x{width3}):", perimeter3)
```

```
Area of Rectangle (5x10): 50
Perimeter of Rectangle (5x10): 30
Area of Rectangle (7x12): 84
Perimeter of Rectangle (7x12): 38
Area of Rectangle (10x15): 150
Perimeter of Rectangle (10x15): 50
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

```python
# --- Code for Legacy Approach (for timing) ---
legacy_code = """
local_names = ["Alice", "Bob", "Charlie", "David"]
local_search_names = ["Charlie", "Eve", "Bob"]
for s in local_search_names:
    found = False
    for n in local_names:
        if s == n:
            found = True
            break
    _ = found # Avoid printing during timing
"""

# --- Code for Optimized Solution (using sets, for timing) ---
optimized_code = """
local_names = ["Alice", "Bob", "Charlie", "David"]
local_search_names = ["Charlie", "Eve", "Bob"]
names_set = set(local_names)
for s in local_search_names:
    _ = s in names_set # Just perform the check, avoid printing during timing
"""

print("--- Performance Comparison (100,000 iterations) ---")

time_legacy = timeit.timeit(legacy_code, number=100000)
print(f"Legacy code execution time: {time_legacy:.6f} seconds")

time_optimized = timeit.timeit(optimized_code, number=100000)
print(f"Optimized code execution time: {time_optimized:.6f} seconds")

print("\n--- Optimized Solution Output ---")
# Actual execution of the optimized solution to show the results
names_set = set(names)
for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

```
--- Performance Comparison (100,000 iterations) ---
Legacy code execution time: 0.047445 seconds
Optimized code execution time: 0.034113 seconds

--- Optimized Solution Output ---
Charlie is in the list
Eve is not in the list
Bob is in the list
```

Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

```python
def calculate_total(price):
    """
    Calculates the tax and total price for a given base price.

    Args:
        price (int or float): The base price of the item.

    Returns:
        float: The total price after adding tax.
    """
    tax_rate = 0.18
    tax = price * tax_rate
    total = price + tax
    return total

# Refactored code using the new function

# Example 1
price1 = 250
total1 = calculate_total(price1)
print(f"Total Price for {price1}:", total1)

# Example 2
price2 = 500
total2 = calculate_total(price2)
print(f"Total Price for {price2}:", total2)
```

```
Total Price for 250: 295.0
Total Price for 500: 590.0
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)

Expected Outcome

• Constants such as PI and RADIUS

• Cleaner, maintainable code

```
import math

# Define constants
PI = math.pi # Using math.pi for a more precise value of Pi
RADIUS = 7

# Calculate and print area and circumference using constants
print("Area of Circle:", PI * (RADIUS ** 2))
print("Circumference of Circle:", 2 * PI * RADIUS)

Area of Circle: 153.93804002589985
Circumference of Circle: 43.982297150257104
```

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

a = 10

b = 20

c = a * b / 2

print(c)

Expected Outcome

• Descriptive variable names

- Inline comments explaining logic
- Identical output

```python
# Define the base and height of the triangle
base = 10  # Length of the triangle's base
height = 20 # Height of the triangle

# Calculate the area of the triangle
area = base * height / 2 # Formula for the area of a triangle

# Print the calculated area
print("Area of the triangle:", area)
```

```
Area of the triangle: 100.0
```